

南开大学

汇编语言与逆向技术课程实验报告

实验三：bubble_sort



学院：密码与网络空间安全学院

专业：信息安全

学号：2412266

姓名：杨滢

班级：信息安全

1 实验时间

第 9 周星期一 9-10 节

2 实验目的

1. 熟悉汇编语言的整数数组；
2. 熟悉基址变址操作数、相对基址变址操作数；
3. 掌握排序算法的底层实现细节

3 实验环境

MASM32 编译环境、Windows 命令行窗口

4 实验过程思路

1. 读取用户输入的数字字符串
2. 解析字符串为 10 个整数存入数组
3. 对数组进行冒泡排序
4. 输出排序后的数组
5. 等待用户按回车键后退出

5 源代码及说明

5.1 源代码及注释

```
1 .386
2 .model flat, stdcall
3 option casemap :none
4
5 ; 包含头文件和库
6 include \masm32\include\windows.inc
7 include \masm32\include\masm32.inc
8 include \masm32\include\kernel32.inc
9 includelib \masm32\lib\masm32.lib
10 includelib \masm32\lib\kernel32.lib
11
12 ; 数据段定义
13 .data
```

```
14 ; 输入缓冲区 - 存储用户输入的原始字符串
15 input_buffer db 100 dup(0) ; 输入缓冲区, 100字节初始化为0
16 output_buffer db 100 dup(0) ; 输出缓冲区, 用于构建输出字符串
17 temp_buffer db 20 dup(0) ; 临时转换缓冲区, 用于数字转字符串
18 newline db 13, 10, 0 ; 换行符和结束符
19 space db " ", 0 ; 空格分隔符和结束符
20
21 ; 数组定义 - 存储解析后的整数
22 array dd 10 dup(0) ; 存储10个32位整数, 每个占4字节
23 array_size equ 10 ; 数组大小常量定义
24
25 ; 代码段定义
26 .code
27
28 ; 主程序入口点
29 start:
30 ; 读取用户输入
31 ; StdIn函数: 从标准输入读取字符串
32 ; 参数1: 缓冲区地址, 参数2: 缓冲区大小
33 invoke StdIn, addr input_buffer, sizeof input_buffer
34
35 ; 解析输入字符串为整数数组
36 call ParseInput
37 cmp eax, 0 ; 检查解析返回值
38 jne exit_program ; 如果解析失败, 直接退出
39
40 ; 执行冒泡排序
41 call BubbleSort
42
43 ; 输出排序后的数组
44 call OutputArray
45
46 ; 等待用户按回车键, 防止程序立即退出
47 invoke StdIn, addr input_buffer, sizeof input_buffer
48
49 exit_program:
50 ; 退出程序
51 invoke ExitProcess, 0
52
53
54 ; ParseInput : 解析输入字符串为整数数组
55 ParseInput proc
```

```
56     local token_count:dword      ; 局部变量: 已解析的数字计数
57
58     mov token_count, 0            ; 初始化计数器为0
59     mov esi, offset input_buffer ; ESI指向输入缓冲区开始
60     mov edi, offset array        ; EDI指向数组开始
61
62 parse_loop:
63     ; 跳过后导空格和分隔符
64 skip_spaces:
65     mov al, [esi]                 ; 加载当前字符到AL
66     cmp al, 0                    ; 检查是否字符串结束
67     je check_count               ; 如果是, 跳转到计数检查
68     cmp al, ' '                  ; 检查是否空格
69     je skip_char
70     cmp al, 13                   ; 检查是否回车符
71     je skip_char
72     cmp al, 10                   ; 检查是否换行符
73     je skip_char
74     cmp al, ','                  ; 检查是否逗号
75     je skip_char
76     jmp found_number             ; 找到有效数字字符
77
78 skip_char:
79     inc esi                       ; 移动到下一个字符
80     jmp skip_spaces
81
82 found_number:
83     ; 找到数字开始位置
84     mov ebx, esi                 ; EBX保存数字字符串的开始地址
85
86     ; 查找数字结束位置
87 find_end:
88     mov al, [esi]
89     cmp al, 0                    ; 字符串结束
90     je process_number
91     cmp al, ' '                  ; 空格分隔符
92     je process_number
93     cmp al, 13                   ; 回车符
94     je process_number
95     cmp al, 10                   ; 换行符
96     je process_number
97     cmp al, ','                  ; 逗号分隔符
```

```
98     je process_number
99     inc esi                      ; 继续查找
100    jmp find_end
101
102 process_number:
103     ; 保存结束字符到CL
104     mov cl, [esi]
105     push ecx                    ; 保存ECX
106
107     ; 在数字结束位置插入结束符, 创建C风格字符串
108     mov byte ptr [esi], 0
109
110     ; 调用MASM32库函数将字符串转换为整数
111     ; atodw函数: ASCII字符串转DWORD整数
112     invoke atodw, ebx           ; 参数: 字符串地址
113
114     ; 检查转换结果是否有效 (0-10000范围内)
115     cmp eax, 0
116     jl conversion_done         ; 如果小于0, 跳过存储
117     cmp eax, 10000
118     jg conversion_done         ; 如果大于10000, 跳过存储
119
120     ; 存储有效数字到数组
121     mov [edi], eax              ; 将转换后的数字存入数组
122     add edi, 4                  ; 移动到数组下一个位置 (每个整数4字节)
123     inc token_count             ; 增加成功解析的计数
124
125 conversion_done:
126     ; 恢复结束位置的原始字符
127     pop ecx
128     mov [esi], cl
129
130     ; 检查是否已解析完10个数字
131     cmp token_count, array_size
132     jge parse_success          ; 如果已解析10个, 成功返回
133
134     ; 移动到下一个字符继续解析
135     inc esi
136     jmp parse_loop
137
138 check_count:
139     ; 检查是否成功解析了10个数字
```

```
140     cmp token_count, array_size
141     je parse_success
142
143 parse_error:
144     mov eax, -1                ; 失败返回-1
145     ret
146
147 parse_success:
148     mov eax, 0                ; 成功返回0
149     ret
150 ParseInput endp
151
152 ; BubbleSort : 冒泡排序算法实现
153
154 BubbleSort proc
155     mov ecx, array_size        ; ECX = 数组大小(10)
156     dec ecx                   ; 外循环次数 = 9
157     jle sort_done             ; 如果数组大小<=1, 直接返回
158
159 outer_loop:
160     mov edx, 0                ; EDX作为交换标志: 0=无交换, 1=有交换
161     mov esi, offset array      ; ESI指向数组开始
162     push ecx                   ; 保存外循环计数器
163
164 inner_loop:
165     ; 比较相邻的两个元素
166     mov eax, [esi]             ; EAX = 当前元素array[i]
167     mov ebx, [esi+4]           ; EBX = 下一个元素array[i+1]
168
169     cmp eax, ebx
170     jle no_swap                ; 如果array[i] <= array[i+1], 不交换
171
172     ; 交换元素 - 将较大的元素向后移动
173     mov [esi], ebx             ; array[i] = array[i+1]
174     mov [esi+4], eax           ; array[i+1] = array[i]
175     mov edx, 1                 ; 设置交换标志
176
177 no_swap:
178     add esi, 4                 ; 移动到下一个元素 (+4字节)
179     loop inner_loop            ; 继续内循环
180
181     pop ecx                    ; 恢复外循环计数器
```

```
182
183     ; 优化: 如果本轮没有发生交换, 说明数组已排序完成
184     test edx, edx                ; 测试交换标志
185     jz sort_done                ; 如果无交换, 提前结束排序
186
187     loop outer_loop             ; 继续下一轮外循环
188
189 sort_done:
190     ret
191 BubbleSort endp
192
193 ; OutputArray : 输出排序后的数组
194 OutputArray proc
195     mov esi, offset array        ; ESI指向数组开始
196     mov ecx, array_size         ; ECX = 数组元素个数
197
198     ; 清空输出缓冲区
199     mov edi, offset output_buffer
200     mov byte ptr [edi], 0        ; 设置字符串结束符
201
202 output_loop:
203     ; 将当前整数转换为字符串
204     push ecx                     ; 保存循环计数器
205     invoke dwtoa, [esi], addr temp_buffer ; dwtoa: DWORD转ASCII字符串
206
207     ; 将转换后的字符串添加到输出缓冲区
208     invoke szCatStr, addr output_buffer, addr temp_buffer ; 字符串连接
209
210     ; 检查是否为最后一个元素
211     pop ecx
212     dec ecx                     ; 计数器减1
213     jz no_space                 ; 如果为最后一个元素, 不加空格
214
215     ; 添加空格作为分隔符
216     push ecx
217     invoke szCatStr, addr output_buffer, addr space
218     pop ecx
219
220 no_space:
221     add esi, 4                  ; 移动到数组下一个元素
222     test ecx, ecx               ; 检查循环是否完成
223     jnz output_loop             ; 如果ECX 0, 继续循环
```

```
224
225     ; 输出最终结果
226     invoke StdOut, addr output_buffer ; 输出排序结果
227     invoke StdOut, addr newline      ; 输出换行
228     ret
229 OutputArray endp
230
231 end start
```

5.2 关键代码分析

1. 基址变址和相对基址变址基址变址 (base-index) 操作数把两个寄存器的值相加, 得到一个偏移地址。两个寄存器分别称为基址寄存器 (base) 和变址寄存器 (index)。; 基址变址 (base-index) 操作数把两个寄存器的值相加, 得到一个偏移地址。两个寄存器分别称为基址寄存器 (base) 和变址寄存器 (index)。格式为 [base + index], 例如 `mov eax, [ebx + esi]`。

```
1  mov eax, [esi]           ; 当前元素 array[i]
2  mov ebx, [esi+4]         ; 下一个元素 array[i+1]
```

这里使用的是相对基址变址寻址:

- 基址寄存器: ESI - 指向数组当前元素的基地址
- 位移量: +4 - 固定偏移量, 因为每个整数占 4 字节
- 格式: [base + displacement] 即 [ESI + 4]

2. 冒泡排序算法优化

```
1  test edx, edx
2  jz  sort_done
```

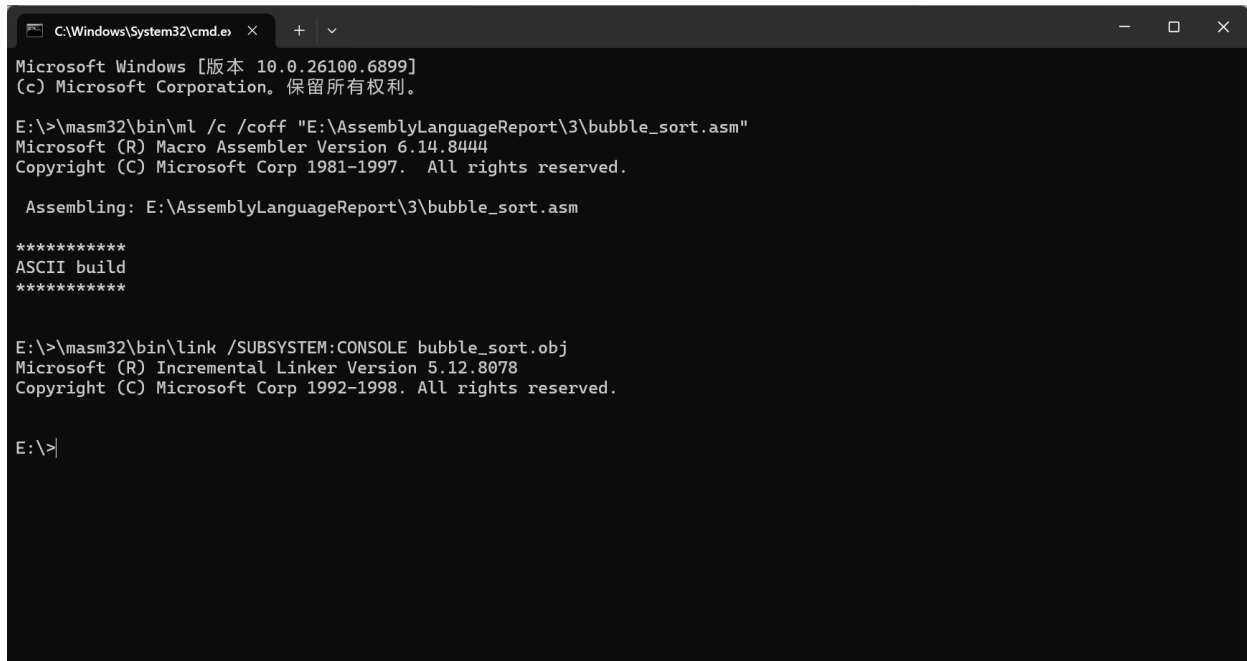
如果某一轮遍历中没有发生任何交换, 说明数组已经有序, 可以提前终止排序。

3. 字符串处理

- 临时字符串终止: 在解析时临时插入 0 字符来创建子字符串
- 库函数利用: 使用 `atodw` 和 `dwtoa` 进行数字与字符串的转换
 - `atodw` 函数: 把 ASCII 字符串转换为 DWORD 整数
 - `dwtoa` 函数: 把 DWORD 整数转换为 ASCII 字符串
- 缓冲区管理: 使用 `szCatStr` 进行字符串拼接

6 bubble_sort.exe 测试过程和截图

6.1 bubble_sort.asm 编译和链接过程说明



```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26100.6899]
(c) Microsoft Corporation。保留所有权利。

E:\>\masm32\bin\ml /c /coff "E:\AssemblyLanguageReport\3\bubble_sort.asm"
Microsoft (R) Macro Assembler Version 6.14.8444
Copyright (C) Microsoft Corp 1981-1997. All rights reserved.

Assembling: E:\AssemblyLanguageReport\3\bubble_sort.asm

*****
ASCII build
*****

E:\>\masm32\bin\link /SUBSYSTEM:CONSOLE bubble_sort.obj
Microsoft (R) Incremental Linker Version 5.12.8078
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.

E:\>|
```

图 6.1: 编译链接

- `\masm32\bin\ml /c /coff "E:\AssemblyLanguageReport3\bubble_sort.asm"`

编译: 将汇编源文件 bubble_sort.asm 翻译成目标文件 (.obj)

- `\masm32\bin\ml`: 调用 MASM 的汇编器程序 ml.exe
- `/c`: 表示只进行汇编, 不自动调用链接器, 即仅生成.obj 文件
- `/coff`: 指定输出的目标文件格式为 COFF
- `"E:\AssemblyLanguageReport3\bubble_sort.asm"`: 输入的汇编源代码文件地址

- `\masm32\bin\link /SUBSYSTEM:CONSOLE bubble_sort.obj`

链接: 将目标文件 bubble_sort.obj 与必要的库文件合并生成最终的可执行文件 (.exe)

- `\masm32\bin\link`: 调用 Microsoft 的 32 位链接器 link.exe
- `/SUBSYSTEM:CONSOLE`: 指定程序运行环境为控制台
- `bubble_sort.obj`: 要链接的目标文件

6.2 bubble_sort.exe 的测试说明

C:\Users\ASUS>start E:\bubble_sort.exe

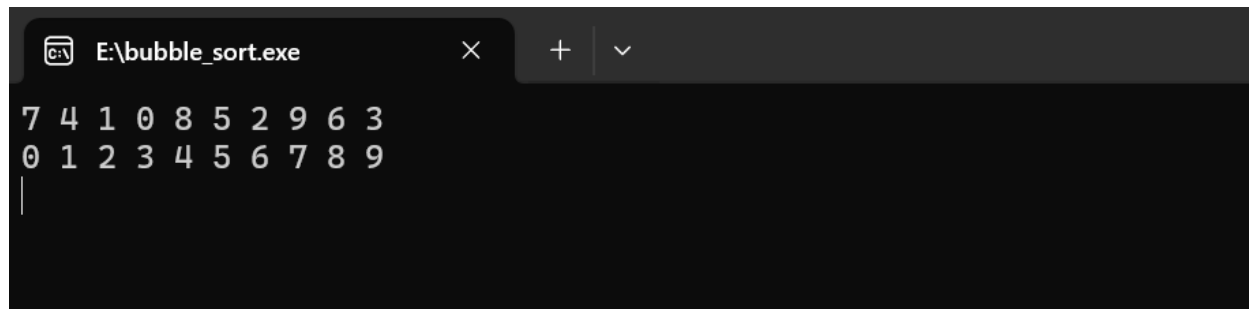


图 6.2: 测试截图 1(空格分割)

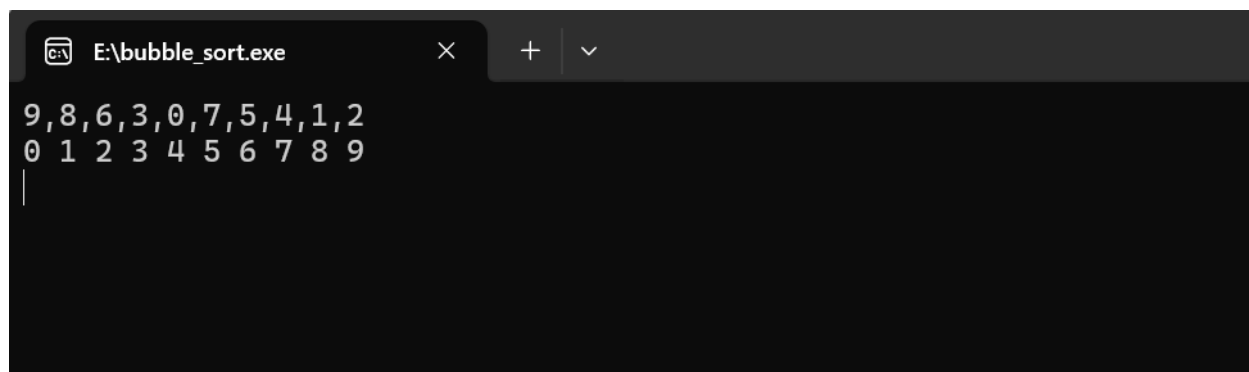


图 6.3: 测试截图 2(英文逗号分割)

测试案例通过。

7 汇编语言数组操作知识点的总结

7.1 数组定义

```
1 ; 一维数组定义
2 array dd 10 dup(0)           ; 10个双字数组, 初始化为0
3 byte_array db 20 dup(?)      ; 20个字节数组, 未初始化
4 word_array dw 5, 10, 15, 20, 25 ; 字数组, 带初始值
5
6 ; 二维数组定义 (模拟)
7 matrix dd 3 * 4 dup(0)       ; 3行4列二维数组
```

7.2 数组寻址

7.2.1 直接寻址

```
1 mov eax, array[0]           ; 访问第一个元素
2 mov ebx, array[4]           ; 访问第二个元素 (双字数组)
```

7.2.2 寄存器间接寻址

```
1 mov esi, offset array      ; ESI指向数组首地址
2 mov eax, [esi]             ; 访问当前元素
3 mov ebx, [esi+4]           ; 访问下一个元素
```

7.2.3 基址变址寻址

```
1 ; 格式: [base + index]
2 mov ebx, offset array      ; EBX = 数组基地址 (基址寄存器)
3 mov esi, 2                 ; ESI = 元素索引 (变址寄存器)
4 mov eax, [ebx + esi*4]     ; 访问 array[2]
```

7.2.4 相对基址变址寻址

```
1 ; 常见格式: [base + index + displacement] 、 displacement[base + index]
2 mov ebx, offset array      ; 基址寄存器
3 mov esi, 1                 ; 变址寄存器
4 mov eax, [ebx + esi*4 + 8] ; 访问 array[1+2] = array[3]
5 mov edx, 4[ebx + esi*4]    ; 另一种格式
```

7.3 数组遍历

7.3.1 指针

```
1 mov esi, offset array      ; 初始化指针
2 mov ecx, array_size        ; 循环计数器
3 traverse_loop:
4     mov eax, [esi]          ; 访问当前元素
5     add esi, 4              ; 移动到下一个元素
6     loop traverse_loop     ; 另一种格式
```

7.3.2 索引

```
1 mov esi, 0                 ; 初始化索引
2 mov ecx, array_size
3 index_loop:
4     mov eax, array[esi*4]   ; 通过索引访问元素
5     inc esi                 ; 增加索引
6     loop index_loop
```

7.3.3 基址变址

```
1 mov ebx, offset array      ; 固定基址
2 mov esi, 0                 ; 变址寄存器
3 mov ecx, array_size
4 base_index_loop:
5     mov eax, [ebx + esi*4]   ; 基址变址寻址
6     inc esi
7     loop base_index_loop
```

8 实验结论及心得体会

8.1 实验结论

bubble_sort 该程序可以实现整数数组的冒泡排序。

8.2 心得体会

1. 经过这次实验,我掌握了汇编语言数组的基础知识,熟悉了汇编语言的整数数组。
2. 不同于 c++ 语言,汇编语言有独特的基址变址寻址和相对基址变址寻址。这次实验让我在实操运用中学习了这部分知识。
3. 另外,在这次实验过程中我还巩固了冒泡排序的相关知识。